

TRABAJO PRÁCTICO 1

Camila Soledad Farías



RESUMEN

Para el desarrollo de las consignas del trabajo decidí que mi implementación se adapte a todos los ejercicios propuestos al mismo tiempo. Cada clase contiene todo lo necesario para la batalla final, por ejemplo los personajes, ya sean magos o guerreros, tienen disponible un arreglo (tipo vector) que puede contener la cantidad de armas que se quiera. A la hora de pelear deberá usar una de ellas, y al crear mis personajes en los ejercicios 2 y 3 me restrinjo a otorgarles únicamente dos armas, aunque mi implementación permita que posean más. Además, utilizo siempre unique pointers para que se sostenga la relación de composición entre los personajes y las armas (las armas componen a los personajes). Lo importante para sostener la composición es que una vez que almaceno un arma dentro de un personaje, esta deje de existir fuera de él; necesito que la única referencia existente al arma este dentro del personaje. Podría controlarlo con shared pointers, eliminando la referencia externa, pero este tipo de puntero no me asegura que no vaya a existir una referencia externa además de la interna (quiero decir que es posible que existan dos punteros si se quiere, pero unique no lo permite). De esta forma, un unique pointer transfiere el ownership del arma al personaje, que es lo que se busca.

EJERCICIO 1

Para el ejercicio uno, decidí incluir en las interfaces tanto de personajes (**IPersonaje**) como de armas (**IArma**) los métodos esenciales necesarios durante la batalla. Esta decisión me permitió aprovechar el polimorfismo, ya que pude definir estos métodos en las clases abstractas, o en sus derivadas, y acceder a ellos correctamente a través de punteros a las interfaces.

Le di mayor importancia a que cada clase derivada de la abstracta tenga un método propio antes de que el mismo sea útil. Agregue un método específico para cada clase derivada aunque todos siguen la misma lógica. Se trata de una frase propia de cada personaje, que podría ser utilizada si ese personaje gana. Este método lleva el nombre de la frase de victoria como se puede ver en la Figura 1.

En el caso de las armas agregué un ataque especial, propio de cada una que, si bien no está aclarado, a la hora de usarlos debería descontarse más puntos de HP al oponente, ya que estos ataques desgastan más el arma. Los mismos modifican un atributo que poseen las armas que indica la cantidad de veces que fueron utilizadas, usarlos le puede costar entre uno y tres usos al arma, dependiendo de cada uno.

Hay niveles para cada tipo de arma. Por un lado los Ítems Mágicos tienen “procedencia”, y las Armas de Combate tienen “material”. En común, las armas tienen un nivel de poder.

Todos estos niveles están definidos en enums y me permiten calcular el daño del arma. Por ejemplo, el daño de un Ítem Mágico es 10 (porque en el ejercicio 3 requiere un daño mínimo de 10 puntos) más el daño extra propio del arma, que se calcula promediando la procedencia con el nivel de poder: $\text{daño total} = \left(\frac{\text{procedencia} + \text{nivel de poder}}{2} \right) + 10$. El cálculo en el caso de

las Armas de combate es el mismo intercambiando procedencia por material (el valor de cada nivel está asignado como un int en el enum).

```
81
82 void Hechicero::show_info() const{
83     cout<<"=== [Hechicero \U0001F9D9] ==="<<endl
84     << "Nombre: " <<name<<endl
85     << "Habilidad Especial: " <<specialHability<<endl
86     << "\u2728 Nivel de Magia: Apprentice" <<endl
87     << "\u2764 HP: " <<HP<<endl;
88 }
89
90 string Hechicero::arcaneSupremacySaying(){
91     return "El poder arcano nunca falla.";
92 }
93
```

Figura 1: Cada clase derivada, en este caso se trata de personajes, tiene su canto/frase de victoria y su propia definición del método `show_info()` que proviene de la interfaz de personajes.

Se definen como static para que no sea necesario un constructor de la clase. De esta forma nunca se instancia **PersonajeFactory** cuando se quieren usar sus métodos.

Además, creé un archivo .hpp que tiene algunas funciones para generar números aleatorios entre un mínimo y un máximo. Este me ayuda para el ejercicio 3 y cumple la consigna del ejercicio 2.

Cómo compilar y ejecutar los archivos del ejercicio 2:

Desde  **Ejercicio2**, ejecutar el siguiente comando en terminal (para compilar)...

```
g++ -Wall CharacterGenerator.cpp PersonajeFactory.cpp main_test.cpp
../Ejercicio1/Arma/ArmaDeCombate.cpp ../Ejercicio1/Personaje/Mago.cpp
../Ejercicio1/Arma/ItemMagico.cpp ../Ejercicio1/Personaje/Guerrero.cpp -o
test
```

luego (para ejecutar),

```
./test
```

EJERCICIO 3

En este ejercicio desarrollé la batalla. Utilizo todas las clases de los ejercicios 1 y 2 para crear el combate entre dos personajes que portan cierta arma. **PersonajeFactory** me permite crear el personaje del jugador y el personaje aleatorio que será su oponente. El personaje del jugador se crea a partir del arma que él elige, la cual es guardada en un arreglo de unique pointers que luego pasa a ser parte del personaje, y a partir del tipo de personaje que elige. Para mostrar la batalla por consola se utiliza el método `show_info()` propio de los personajes.

Cómo compilar y ejecutar los archivos del ejercicio 3:

Desde  **Ejercicio3**, ejecutar el siguiente comando en terminal (para compilar)...

```
g++ -Wall main.cpp ../Ejercicio2/PersonajeFactory.cpp
../Ejercicio2/CharacterGenerator.cpp ../Ejercicio1/Personaje/Guerrero.cpp
../Ejercicio1/Personaje/Mago.cpp ../Ejercicio1/Arma/ArmaDeCombate.cpp
../Ejercicio1/Arma/ItemMagico.cpp -o ejec
```

luego (para ejecutar),

```
./ejec
```

APRENDIZAJES

Haciendo este trabajo aprendí varias cosas, entre ellas a acceder a archivos que se encuentran fuera del directorio en el que estoy trabajando, utilizando ../ para salir de la carpeta y luego agregando el path correspondiente. También me quedó claro por qué estaría mal indicar que hay una relación de composición entre dos interfaces, pues estas no tienen atributos y no hay forma de que una componga a la otra; si existe una relación es entre sus derivadas, y así es como lo expreso en el UML, no se usa el rombo de composición sino una flecha que indica que las clases que hereden de las abstractas también heredarán dicha relación.

Después, logré trabajar únicamente con unique pointers. En una versión previa del trabajo había usado raw pointers dentro de un método de las clases **Mago** y **Guerrero**, pero entendí que la idea de polimorfismo es en parte facilitar el manejo de las clases pudiendo acceder a métodos que se definen distinto para cada una. De esta manera, note que podía incluir este método en la interfaz **IPersonaje** y ya no tendría la necesidad de usar un raw pointer para hacer dynamic_cast de los personajes identificando si eran magos o guerreros.